

---

name: tdd description: Test-driven development implementation skill. Enforces strict Red-Green-Refactor cycles, writing deterministic unit tests before executing any production code.

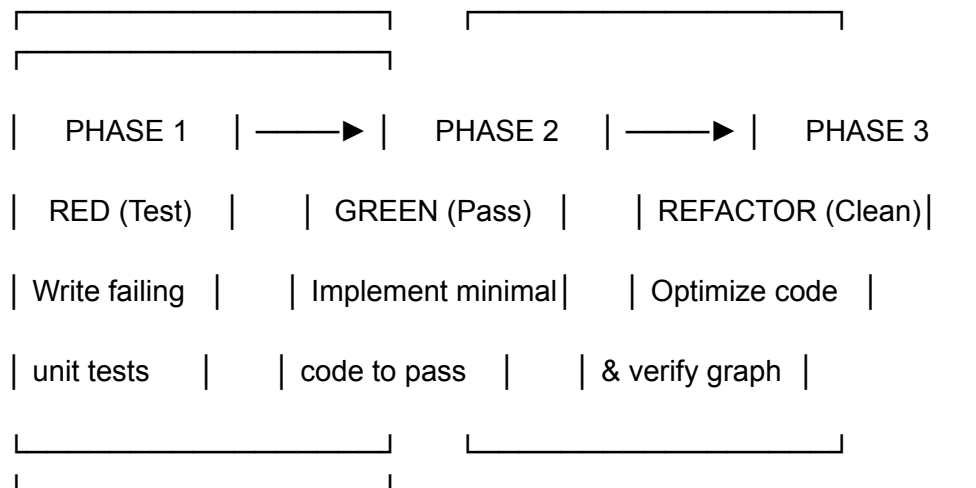
## TDD — Test-Driven Development Implementation Skill

### Purpose & Protocol

The **TDD** skill enforces that no functional code is written until a comprehensive, failing automated test exists. This guarantees 100% verifiable behavior, prevents regressions, and produces self-certifying artifacts.

---

### The Three-Phase Cycle



### Phase 1: RED (Author Failing Tests)

1. Read the target ticket and relevant architectural spec.
2. Create test files in `tests/` or alongside the module.
3. Write test cases covering:
  - Happy path (valid inputs, expected outputs).
  - Boundary conditions (empty payloads, max token limits, edge strings).

- Error handling (malformed JSON, broken socket connections, missing files).
- 4. Run test runner (`pytest`, `npm test`, or custom bash runner).
- 5. **Verify Failure:** Confirm tests fail for the *expected reasons* (e.g., `ModuleNotFoundError` or `AssertionError`), not syntax errors in the test suite itself.

#### Phase 2: GREEN (Minimal Implementation)

1. Implement the simplest, most direct code that satisfies the failing assertions.
2. Do not prematurely optimize or add speculative features outside the ticket scope.
3. Run test runner again until all test cases pass with exit code 0.

#### Phase 3: REFACTOR (Hygiene & Verification)

1. Clean up variable names, remove debug logging, and ensure deterministic formatting.
2. Verify that no AST cycles or lint violations were introduced (`graph_builder.py`).
3. Re-run the full test suite to guarantee zero regressions.